

PYTHON FOR AI & MACHINE LEARNING

PERSONAL STUDY MATERIAL

A beginner-friendly, practical learning guide

LEARN THE BASICS • WRITE CODE • BUILD MINI-PROJECTS

Purpose of this guide

This course introduces Python programming from the ground up, then connects those skills to the foundations of Artificial Intelligence (AI) and Machine Learning (ML). It includes explanations, runnable code examples, practice tasks, mini-projects, and an answer key.

How to study

1. Type each example yourself instead of only reading it.
2. Predict the output before running the code.
3. Change one part and observe what happens.
4. Complete the exercises without looking at the answer key.
5. Save your programs and revise them regularly.

Audience: complete beginners and students preparing for introductory AI/ML study. Examples use standard Python 3 syntax.

Contents & Learning Outcomes

Module	Topics
1. Getting started	What Python is, setup, first program, comments, errors
2. Variables & data types	Names, numbers, strings, booleans, type conversion
3. Operators & input/output	Calculations, comparisons, input(), print(), f-strings
4. Decisions & loops	if/elif/else, for, while, range(), break, continue
5. Collections	Lists, tuples, sets, dictionaries, indexing, slicing
6. Functions & modules	Parameters, return, scope, imports
7. Errors, files & debugging	Exceptions, try/except, reading and writing files
8. Python for data & AI	NumPy, pandas, plotting, ML workflow, scikit-learn
9. Practical projects	Grade calculator, study tracker, simple ML classifier
10. Revision	Cheat sheet, exercises, answers, 14-day plan

Learning outcomes

- Write and run short Python programs.
- Use variables, conditions, loops, collections, and functions.
- Read simple errors and debug programs methodically.
- Work with basic data tables and understand the role of Python libraries in AI.
- Explain the main stages of a simple machine-learning workflow.

1. Getting Started with Python

Python is a general-purpose programming language known for readable syntax and a large ecosystem of libraries. It is used in automation, websites, data analysis, scientific computing, AI, and ML.

Ways to run Python

- Install Python 3 from the official Python website and use an editor or IDE.
- Use a notebook environment for experiments and data analysis.
- Use a terminal to run a saved file, for example: **python hello.py** (the command may be **python3** on some systems).

Your first program

```
print("Hello, world!")
```

The **print()** function displays information. Text is written inside quotation marks. Python is case-sensitive: **name** and **Name** are different names.

Comments and indentation

```
# This is a comment
print("Python is readable")

if 4 > 2:
    print("Indentation defines the block")
```

Comments begin with # and are ignored by the interpreter. Indentation is meaningful in Python; use consistent spacing, commonly four spaces per level.

Common beginner errors

Error	Typical cause	How to fix it
SyntaxError	Invalid Python structure, missing colon, or unmatched quote.	Check punctuation and the line reported.
NameError	Using a name that has not been defined.	Check spelling and where the variable is created.
TypeError	Using a value in an unsupported way.	Check the types of the values and convert if appropriate.
IndentationError	Inconsistent or missing indentation.	Align statements inside each block.

Try it

Write a program that prints your favourite subject, one skill you want to learn, and the sentence "I am learning Python for AI." Do not include private personal details in shared notebooks.

2. Variables & Data Types

A variable is a name that refers to a value. In Python, you create or update a variable with the assignment operator `=`.

```
student = "Asha"
age = 18
score = 92.5
is_enrolled = True

print(student)
print(age)
print(type(score))
```

Common built-in types

Type	Example	Use
int	count = 4	Whole numbers
float	temperature = 28.5	Numbers with a decimal part
str	course = "Python"	Text
bool	is_ready = False	True/False values
NoneType	result = None	Represents no value / not set

Naming variables

- Use descriptive names such as **total_marks** rather than vague names like **x**.
- Names can contain letters, digits, and underscores, but cannot start with a digit.
- Python keywords such as **if**, **for**, and **class** cannot be used as variable names.
- By convention, Python variable and function names use lowercase words separated by underscores.

Type conversion

```
age_text = "18"
age_number = int(age_text)
price = float("19.99")
score_text = str(95)

print(age_number + 2) # 20
```

Conversions can fail if the text is not a valid representation, such as `int("hello")`.

Strings

```
first = "Ada"
last = "Lovelace"
full_name = first + " " + last
print(full_name)
print(len(full_name))
print(full_name.upper())
```

Strings are sequences of characters. Useful operations include `len()`, `.lower()`, `.upper()`, and slicing such as `word[0:3]`.

3. Operators, Input & Output

Arithmetic operators

Operator	Meaning	Example result
+	Addition	$7 + 2 \rightarrow 9$
-	Subtraction	$7 - 2 \rightarrow 5$
*	Multiplication	$7 * 2 \rightarrow 14$
/	Division	$7 / 2 \rightarrow 3.5$
//	Floor division	$7 // 2 \rightarrow 3$
%	Remainder	$7 \% 2 \rightarrow 1$
**	Power	$7 ** 2 \rightarrow 49$

Comparison and logical operators

```
age = 18
print(age >= 18) # True
print(age < 25)  # True
print(age >= 18 and age < 25) # True
print(not False) # True
```

Comparisons include `==`, `!=`, `<`, `<=`, `>`, and `>=`. Logical operators are **and**, **or**, and **not**. Remember: `=` assigns a value, while `==` compares values.

Getting input

```
name = input("Enter your name: ")
print(f"Welcome, {name}!")

age = int(input("Enter your age: "))
print(f"Next year you will be {age + 1}.")
```

Important: `input()` returns text. Convert it with `int()` or `float()` when you need numeric calculations.

Formatted strings (f-strings)

```
subject = "AI"
marks = 87
print(f"My subject is {subject} and my marks are {marks}.")
```

Mini exercise

Ask the user for two numbers and print their sum, difference, product, and quotient. Decide what your program should do if the second number is zero.

4. Decisions & Loops

Conditional statements

```
marks = int(input("Enter marks (0-100): "))

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Needs more practice"

print(f"Result: {grade}")
```

Python checks conditions from top to bottom and runs the first matching branch. Validate input ranges in real programs; this example assumes marks are between 0 and 100.

for loops

```
for number in range(1, 6):
    print(number)

subjects = ["Maths", "Science", "Python"]
for subject in subjects:
    print(subject)
```

range(1, 6) produces 1, 2, 3, 4, 5; the ending value is excluded.

while loops

```
count = 3
while count > 0:
    print(count)
    count -= 1
print("Go!")
```

A while loop repeats while its condition is true. Make sure the loop can eventually stop to avoid an infinite loop.

break and continue

```
for n in range(1, 8):
    if n == 5:
        break          # stop the loop
    if n % 2 == 0:
        continue       # skip this iteration
    print(n)
```

Mini exercise

Write a loop that prints the numbers from 1 to 20 and labels each as even or odd. Hint: use `number % 2`.

5. Collections: Lists, Tuples, Sets & Dictionaries

Lists

```
subjects = ["Maths", "English", "Python"]
print(subjects[0])          # Maths
subjects.append("AI")
subjects[1] = "Science"
print(len(subjects))
print(subjects[1:3])
```

Lists are ordered and mutable (you can change them). Indexing starts at 0; negative index -1 refers to the last item. Slicing uses a start index and an excluded end index.

Tuples and sets

```
point = (4, 7) # tuple: ordered, generally used as an immutable group
unique_scores = {80, 90, 80, 75}
print(unique_scores) # duplicates are removed
```

A **tuple** is useful for a fixed group of values. A **set** stores unique elements and is useful for membership checks; sets do not guarantee a display order.

Dictionaries

```
student = {
    "name": "Asha",
    "marks": 92,
    "course": "Python"
}
print(student["name"])
student["marks"] = 95
student["city"] = "Kochi"
print(student.get("email", "Not provided"))
```

A dictionary stores key-value pairs. Use **dict[key]** when the key is expected to exist, or **dict.get(key, default)** when it may be missing.

Choosing the right collection

Collection	Best for	Example
list	Ordered items that may change	A list of revision topics
tuple	A fixed group of values	An (x, y) coordinate
set	Unique items and membership checks	Unique course tags
dict	Values looked up by a key	Student record or configuration

Mini exercise

Create a dictionary for a book with title, author, and year. Print the title, update the year, and add a key named available with the value True.

6. Functions & Modules

A function is a reusable block of code. It can accept inputs called parameters and send a result back using **return**.

```
def calculate_average(scores):
    """Return the average of a non-empty list of scores."""
    return sum(scores) / len(scores)

marks = [80, 90, 85]
average = calculate_average(marks)
print(f"Average: {average:.1f}")
```

In this example, **scores** is a parameter, **marks** is the argument passed to the function, and **return** sends the result back to the caller.

Default parameters and validation

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Sam"))
print(greet("Sam", "Good morning"))
```

Variables created inside a function are usually local to that function. Keep functions focused: one clear task per function is a useful beginner habit.

Importing modules

```
import math
print(math.sqrt(81))

from random import randint
print(randint(1, 6))
```

A module is a file or library of reusable Python code. Python includes many standard-library modules, such as **math**, **random**, **datetime**, and **statistics**.

Good programming habits

- Use meaningful names and consistent indentation.
- Break a long program into small functions.
- Add comments when they explain why something is done, not merely repeat the code.
- Test normal inputs, boundary cases, and unexpected inputs.
- Avoid copying code you cannot explain; understand each line.

7. Errors, Files & Debugging

Errors are a normal part of programming. Read the error message, identify the line, check the values and types, then reproduce the problem with a small example.

Handling exceptions

```
try:
    number = int(input("Enter a whole number: "))
    print(f"Double: {number * 2}")
except ValueError:
    print("Please enter a valid whole number.")
```

Use **try/except** for errors you can reasonably handle. Avoid catching every possible exception without a plan, because that can hide bugs.

Reading and writing a text file

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Revise Python basics\n")

with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

The **with** statement closes the file automatically. Mode **w** writes and replaces existing contents; mode **a** appends; mode **r** reads. Be careful when writing to important files.

Debugging checklist

- Read the complete error message and note the line number.
- Check spelling, quotes, brackets, colons, and indentation.
- Print or inspect intermediate values and their types.
- Try the smallest input that reproduces the issue.
- Fix one issue at a time and rerun the program.

Exercise

Modify the input example so that it keeps asking until the user enters a valid integer. Hint: use a while loop and break after successful conversion.

8. Python for Data & AI

Python is widely used in AI because it has libraries for numerical computing, data handling, visualization, and machine learning. Learn core Python first, then add libraries as you need them.

Common libraries

Library	Purpose	Typical use
NumPy	Efficient numerical arrays and operations	Vector and matrix calculations
pandas	Tabular data manipulation	Reading CSV files, filtering rows, summaries
Matplotlib	Charts and plots	Visualizing trends and distributions
scikit-learn	Traditional machine-learning tools	Preprocessing, model training, evaluation
PyTorch / TensorFlow	Deep-learning frameworks	Building and training neural networks

Installation note

In a terminal or notebook environment, libraries are commonly installed with pip, for example: **python -m pip install numpy pandas matplotlib scikit-learn**. Follow your course's environment instructions, and avoid installing packages into a system environment you do not control.

NumPy example

```
import numpy as np

scores = np.array([70, 85, 90, 95])
print(scores.mean())
print(scores.max())
```

pandas example

```
import pandas as pd

data = {
    "subject": ["Maths", "Science", "Python"],
    "marks": [82, 88, 95]
}
df = pd.DataFrame(data)
print(df)
print(df["marks"].mean())
```

A simple machine-learning workflow

- **1. Define:** state the problem and what counts as success.
- **2. Prepare:** collect suitable, permitted data and check its quality.
- **3. Split:** separate training and test data; use validation data or cross-validation for model selection when appropriate.
- **4. Train:** fit a model using training data.
- **5. Evaluate:** measure performance on data not used for fitting.
- **6. Review:** check limitations, fairness, privacy, and whether the model is appropriate for use.

Example: train a tiny classifier

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42,
```

```
        stratify=iris.target
    )
    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    print("Accuracy:", accuracy_score(y_test, predictions))
```

This example uses a built-in educational dataset. Accuracy is only one measure, and a good score on a small demonstration does not prove a model is fair, safe, or ready for real-world use.

9. Beginner Projects

Project 1 — Grade calculator

Goal: calculate an average from several marks and display a result. Start with fixed values, then extend the program to ask the user for input.

```
marks = [80, 76, 91, 88]
average = sum(marks) / len(marks)

if average >= 90:
    result = "Excellent"
elif average >= 75:
    result = "Good progress"
else:
    result = "Keep practising"

print(f"Average: {average:.1f}")
print(result)
```

Challenge: reject marks outside 0–100 and handle an empty list safely.

Project 2 — Study topic tracker

Goal: store revision topics and mark each one as completed or pending. This practices dictionaries and loops.

```
topics = {
    "Variables": False,
    "Loops": True,
    "Functions": False,
    "Lists": True
}

for topic, completed in topics.items():
    status = "Done" if completed else "To revise"
    print(f"{topic}: {status}")

completed_count = sum(topics.values())
print(f"Completed: {completed_count}/{len(topics)}")
```

Challenge: allow the user to mark a topic as complete, and save the tracker to a file.

Project 3 — Data summary

Goal: summarize a list of numeric measurements using built-in tools.

```
import statistics

study_minutes = [25, 40, 30, 45, 35, 50, 20]
print("Days recorded:", len(study_minutes))
print("Average minutes:", statistics.mean(study_minutes))
print("Median minutes:", statistics.median(study_minutes))
print("Longest session:", max(study_minutes))
```

Challenge: load the values from a CSV file and plot them using Matplotlib. Use your own study data only if you are comfortable storing it.

Project checklist

- Can you explain the program without reading the code line by line?
- Have you tested normal, boundary, and invalid inputs?
- Are variable names clear and functions small?
- Does the output match what you expected?
- Have you avoided exposing private information?

10. Quick Revision & Practice

Python quick-reference sheet

Task	Syntax / example
Display output	<code>print("Hello")</code>
Create a variable	<code>count = 10</code>
Get user input	<code>name = input("Name: ")</code>
Convert text to integer	<code>age = int(input("Age: "))</code>
Condition	<code>if score >= 50: ... else: ...</code>
Loop over numbers	<code>for i in range(5): ...</code>
Loop over a list	<code>for item in items: ...</code>
Create a function	<code>def add(a, b): return a + b</code>
Add to a list	<code>items.append(value)</code>
Dictionary lookup	<code>student.get("name", "Unknown")</code>
Handle conversion error	<code>try: ... except ValueError: ...</code>
Import a module	<code>import math</code>

Practice questions

1. What is the difference between `=` and `==`?

Answer: _____

2. What output does `range(2, 6)` produce?

Answer: _____

3. Why does `input("Enter age: ")` need `int()` before numeric addition?

Answer: _____

4. When would you choose a dictionary rather than a list?

Answer: _____

5. What does `return` do in a function?

Answer: _____

6. What is the purpose of a train/test split in machine learning?

Answer: _____

7. Name two checks you should make before trusting an AI model's result.

Answer: _____

Coding challenges

8. Write a function `is_even(number)` that returns `True` when a number is even.

9. Given a list of marks, calculate the highest, lowest, and average mark.

10. Create a dictionary of three subjects and their marks, then print each subject with its mark.

11. Modify the study tracker project to count pending topics as well as completed topics.

Answer Key & Study Plan

Practice answers

- **1.** `=` assigns a value; `==` tests whether two values are equal.
- **2.** 2, 3, 4, 5; the stop value 6 is excluded.
- **3.** `input()` returns a string, so convert valid numeric text to an integer before arithmetic.
- **4.** Use a dictionary when you want to look up values by meaningful keys.
- **5.** `return` sends a result to the code that called the function.
- **6.** To estimate how well a model performs on data it did not use to learn its parameters.
- **7.** Check test-data performance, data quality, possible bias, privacy, limitations, and whether the evaluation fits the task.
- **8.** Example: `def is_even(number): return number % 2 == 0.`
- **9.** Use `max(marks)`, `min(marks)`, and `sum(marks) / len(marks)` after checking the list is not empty.
- **10.** Create a dictionary, then use `for subject, mark in scores.items(): print(subject, mark).`
- **11.** Use `sum(topics.values())` for completed topics and `len(topics) - sum(topics.values())` for pending topics.

A 14-day beginner study plan

Day	Study focus	Practice task
1	Run Python and <code>print()</code>	Write five print statements.
2	Variables and types	Store and print five different values.
3	Strings and conversion	Build a greeting and age calculator.
4	Operators and input	Make a two-number calculator.
5	<code>if / elif / else</code>	Create a grade calculator.
6	<code>for</code> and <code>while</code> loops	Print and classify numbers.
7	Lists and tuples	Store and process a list of marks.
8	Sets and dictionaries	Create a student or book record.
9	Functions	Write three small reusable functions.
10	Errors and files	Handle invalid input and save notes.
11	NumPy / pandas	Create an array and a small DataFrame.
12	ML concepts	Explain features, labels, training, and testing.
13	Mini-project	Build or extend the study tracker.
14	Revision	Answer the questions and explain your code aloud.

My reflection

The Python topic I understand best: _____

The topic I need to practise: _____

My next small project: _____

Final reminder: programming is learned by doing. Write code, run it, read errors calmly, test changes, and explain what you learned. For AI, always verify important results and use data responsibly.